

Assignment 1 Extra

Data Types & Arrays

Organize your code in the proper way to have ordered output, where each of the following questions has its output displayed contiguously, achieve this either using single initial block or multiple initial blocks with proper delays beyond them.

Part A: Scenario Based

For each situation below, you are the verification engineer modeling a signal that comes from a real digital IC block. Choose the single most appropriate SystemVerilog data type (**type, width and sign**) that matches the described hardware, declare it, and justify your choice in a one-line comment:

- FIFO occupancy counter:** A synchronous FIFO reports how many bytes it currently holds (0–255). This is a pure two-state count that **never needs X/Z**. Declare a variable “b” sized exactly for it. 
- DSP accumulator sample:** A digital filter datapath produces a signed accumulator sample that can swing roughly between –32000 and +32000. Declare a variable “c” that stores one such sample.
16 bits signed shortint c;
-2^(N-1) to (2^(N-1))-1
- Decoded opcode field:** A 4-bit instruction opcode is driven by purely combinational logic in your testbench model; it only ever needs the values 0 and 1 (**both X/Z modeling required here**). Declare a variable “d” of the leanest type that fits. logic [3:0] d; means 4 states

Now write an initial block that drives and reports the modeled signals, exactly as the simulator would log them:

- Drive b with an occupancy of 250 and display its value.
- Drive c with the sample -12345 and display it.
- Drive d with 4'b1101 and display it in binary after a 5ns delay.

Part B: Scenario Based

For every item you must decide which SystemVerilog construct matches the described storage and access pattern, declare it with the appropriate element **type, width, sign and dimensions**, and justify your choice in a one-line comment then initialize and print it.

1. **FIR digital filter coefficient bank:** A digital filter has a fixed set of whole number weights (1, 2, 3, 4, 5) that never changes size and is always read one weight at a time in order. Choose storage that holds these five ordered weights, then initialize it, finally display content. In your solution, state whether the elements need to be packed or unpacked and why.

Packed

Packed. > A packed array stores all elements as a single, contiguous block of bits in memory. Since the coefficients are read sequentially, this layout maximizes spatial locality, avoids simulator word-alignment padding, and minimizes access time.

2. Consider the same situation as "1.", the additional detail in the current scenario is that you never need to treat these weights as a single vector of adjacent elements. Now resolve the case, and write in your solution what is the different from case "1.".

Unpacked

3. **Per-channel gain table.** A DAC contains four channels (A, B, C, and D). Each channel stores an independent 8-bit gain value. Channels A and B gains must be stored as one adjacent pair, and channels C and D gains must be stored as a second adjacent pair. The pair (A, B) must not be contiguous with the pair (C,D). Choose an appropriate SystemVerilog data type and initialize the gains to A=10, B=20, C=30, and D=40.

4 places each place has 8 bits

مش فاهم

4. **Register descriptor:** A register file carries three pieces of information that belong together but are of different types: a textual name, a numeric address, and a reset value. You want one storage unit that saves these info's. Decide which construct can group heterogeneous fields under a single type, define it (call the type `reg_t`, and choose an appropriate type for each of the three fields), then:

a) Declare storage for exactly 3 such records, called **regfile**, choosing the array kind suited to a fixed, index-addressed table.

b) Populate it:

Index	Name	Address	Reset value
0	"CTRL"	'h00	8'h01
1	"STATUS"	'h04	8'h00
2	"DATA"	'h08	8'hFF

c) Print each entry showing name, address and reset value, choosing format specifiers that suit each field's type.

Union

5. **Aliased register access:** Hardware lets the same 32-bit memory-mapped register be read two ways that share the same storage: First read way is full 32-bit word, while second read way is just its least-significant byte. Decide which construct lets two differently-typed members occupy the same memory so writing one changes what the other reads. Define it (call the type `reg_view_u`), then:

a) Declare a variable `acc` of that type.

b) Write `32'h12345678` through the word view. **Before you run it**, predict what the byte view will read, then print both views in hex and check.

c) Write `8'h12` through the word view, and print both views in hex again.

d) In a comment, explain from your two observations how the members share storage and why this behaves differently from item 5, where the fields were grouped instead of overlapped.

Part C: Scenario Based

A module monitors the interface for an entire simulation run. It cannot know in advance how many transactions will appear the run might produce five or fifty, decided only as the simulation executes. You must **choose storage that starts sized at run time and can grow and shrink as the run unfolds, declare it with the right element type, and justify the choice in a one-line comment.** Dynamic Array

Deliverables:

You have to submit two separate files

- **One PDF** contains:

- Snippets for the implemented code (story Parts A–C).
- Snippets for the waveform showing the values of the created variables.
- Snippets for the logs showing all printed/displayed values.

- **One Zip file** contains:

- Snippets for the implemented code.
- Snippet for the **run.do** file (that you've used for automating the process).

The delivered zip file must be named like **your_name_Assignment_1_Extra.rar** for example:

Hassan_Khaled_Assignment_1_Extra.rar

Good Luck